Software Code Standard

Project Phoenix · C++17 · PHX-SCS-001

1. Introduction

This document defines the mandatory software coding standards applicable to all source code developed under **Project Phoenix**. Compliance with these standards is required for all software components across all Design Assurance Levels (DAL A, B, and C) in accordance with the objectives of **DO-178C**. The standards herein are established to ensure software safety, reliability, determinism, and long-term maintainability.

Deviations from any rule designated **FORBIDDEN** are not permitted under any circumstance. Deviations from rules designated **Restricted** require documented justification and written approval from the Software Lead and Safety Engineer prior to implementation.

2. Prohibited and Restricted Language Constructs

The following table defines the permissibility of specific C++17 constructs by Design Assurance Level. Rules are cumulative: a construct forbidden at DAL A remains forbidden in all higher-criticality contexts.

Rule ID	Construct	DAL A	DAL B	DAL C	Description
SCS-L-01	**Dynamic Memory** new, delete, malloc, free	FORBIDDEN	FORBIDDEN	Restricted	Dynamic heap allocation introduces non-deterministic timing and fragmentation. All memory shall be statically allocated at compile time or via pre-allocated pools. Pool allocators at DAL C require a documented memory safety analysis.
SCS-L-02	**goto Statement**	FORBIDDEN	FORBIDDEN	FORBIDDEN	The goto statement produces unstructured control flow that cannot be reliably analyzed for coverage or correctness. There are no permitted exceptions at any DAL.
SCS-L-03	**Exceptions** try, catch, throw	FORBIDDEN	FORBIDDEN	Restricted	C++ exception handling introduces non-deterministic stack unwinding and hidden control flow paths. At DAL C, use is restricted to non-safety-critical subsystems with full MC/DC analysis. Compile with -fno-exceptions at DAL A and DAL B.
SCS-L-04	**Recursion**	FORBIDDEN	Restricted	Allowed	Recursive functions preclude static worst-case stack depth analysis. At DAL A, all algorithms shall be implemented iteratively. At DAL B, recursion requires a formal, tool-verified bounded-depth proof in the software design document.

Rule ID	Construct	DAL A	DAL B	DAL C	Description
SCS-L-05	**RTTI** dynamic_cast, typeid	FORBIDDEN	FORBIDDEN	Restricted	RTTI relies on runtime type resolution and introduces non-deterministic overhead. At DAL C, restricted to non-flight-critical diagnostic/logging modules only. Compile with -fno-rtti at DAL A and DAL B.

3. Complexity Metrics

All software functions shall comply with the following structural complexity limits. Functions exceeding an applicable limit shall be refactored prior to code review. Metrics shall be computed using a qualified static analysis tool (e.g., Polyspace, PC-lint Plus) as part of the standard build verification process.

Rule ID	Metric	DAL A Limit	DAL B Limit	DAL C Limit
SCS-M-01	**Cyclomatic Complexity** (McCabe) Max independent paths through a function	≤ 10	≤ 15	≤ 20
SCS-M-02	**Function Length** (executable lines of code) Excluding comments and blank lines	≤ 50	≤ 75	≤ 100
SCS-M-03	**Control Flow Nesting Depth** Loops, conditionals, switch statements	≤ 3	≤ 4	≤ 5
SCS-M-04	**Number of Function Parameters** Including output reference parameters	≤ 4	≤ 5	≤ 6

4. Naming and Documentation Standards

4.1 Naming Conventions

All identifiers shall conform to the following conventions. Consistency is mandatory; mixed conventions within a single translation unit are not permitted.

Element	Convention	Example
Functions	PascalCase	`ComputeAltitudeRate()`
Variables (local & member)	camelCase	`sensorReadingRaw`
Classes & Structs	PascalCase	`NavigationFilter`
Constants & constexpr	SCREAMING_SNAKE_CASE	`MAX_SAMPLE_RATE_HZ`

4.2 Function Header Comment Requirement

Rule SCS-D-01 (Mandatory): Every function definition shall be immediately preceded by a structured header comment block. The comment shall, at minimum, describe the function's purpose, its inputs and outputs, and any preconditions or side effects. A project-standard Doxygen-compatible format is required, as shown below.

```
/** * @brief Computes the filtered altitude rate from raw sensor data. * * @param[in] rawAltitude
Unfiltered altitude measurement, in metres. * @param[in] deltaTimeSec Elapsed time since last
sample, in seconds. * @param[out] rateMetersPerSec Computed altitude rate (m/s). * * @pre
deltaTimeSec shall be greater than zero. * @return PHX_STATUS_OK on success;
PHX_STATUS_ERR_INVALID_INPUT if * preconditions are not satisfied. */ PhxStatus
ComputeAltitudeRate(float rawAltitude, float deltaTimeSec, float& rateMetersPerSec);
```

Absence of a compliant header comment constitutes a **non-conformance finding** during Software Code Review (SCR) and shall block integration of the affected module.

End of Document — PHX-SCS-001 — Project Phoenix Software Code Standard